

MULTI PROCESSING AND THREADING

This is a very common system design and OS interview question.
The interviewer usually wants to know **when to use processes vs threads**, not just the definitions.

Multi-Processing

Multiple **processes** execute concurrently.
They communicate using IPC (pipes, sockets, shared memory, etc.).

Multi-Threading

A **single process** contains multiple threads.

Advantages of Multiprocessing

1. Better Isolation

If Chrome crashes,

Chrome ❌

VS Code ✅

Spotify ✅

Nothing else is affected.

2. Better Security

One process cannot directly access another process's memory.
This prevents accidental corruption and improves security.

3. Uses Multiple CPU Cores

Each process can run on a different core.

Core 1 → Chrome

Core 2 → VS Code

Core 3 → Spotify

4. Easier Debugging

Processes are independent.
A bug in one process is less likely to corrupt another.

Disadvantages of Multiprocessing

High Memory Usage

Every process has:

- Own heap
- Own stack
- Own page table
- Own PCB

This increases memory consumption.

Expensive Context Switching

When switching processes, the OS must:

- Save registers
- Switch address space
- Update page tables
- Flush/reload TLB entries (on many systems)

This is slower than switching threads.

IPC is Slower

Processes cannot simply read each other's variables.

Instead, they use:

- Pipes
- Shared memory
- Message queues
- Sockets

These mechanisms add overhead compared to shared-memory communication within a process.

Advantages of Multithreading

Fast Communication

Threads share memory.

```
globalCounter++;
```

Every thread can access it (with proper synchronization).

No IPC required.

Low Memory Usage

Only one copy of:

- Code
- Heap
- Global variables

is maintained.

Faster Creation

Creating a thread is much cheaper than creating a process because it reuses the existing process resources.

Faster Context Switching

The address space remains the same.

The OS mainly switches:

- Registers
 - Stack Pointer
 - Program Counter
-

Disadvantages of Multithreading

Race Conditions

Suppose:

```
count++;
```

Two threads execute it simultaneously.

Result may be wrong.

Need:

- Mutex
- Semaphore
- Atomic operations

Harder Debugging

Concurrency bugs like:

- Deadlock
- Race condition
- Starvation
- Livelock

can be difficult to reproduce and fix.

Poor Isolation

Suppose Thread A does:

```
int *p = nullptr;

*p = 10;
```

It may crash the entire process because all threads share the same address space.

When to Use Multiprocessing

Use it when you need:

- Strong isolation
- Better fault tolerance
- Security boundaries
- CPU-intensive work that benefits from separate processes

Examples:

- Web browsers (multiple renderer processes)
- Database servers
- Docker containers (containers run processes)
- Separate microservices

When to Use Multithreading

Use it when tasks:

- Share lots of data
- Need fast communication
- Belong to the same application
- Benefit from lower overhead

Examples:

- Web server worker threads
- Game engine (rendering, physics, audio)
- IDE (syntax highlighting, indexing, UI)
- Video encoding
- Chrome's UI, renderer, and networking threads within a process

Interview Answer (2–3 minutes)

Multiprocessing means running multiple independent processes, where each process has its own address space and resources. It provides strong isolation, so if one process crashes, the others continue running. However, process creation, context switching, and communication through IPC are relatively expensive.

Multithreading means running multiple threads within the same process. Threads share the process's memory and resources, making them lightweight and enabling fast communication. Thread creation and context switching are cheaper than processes. However, because threads share memory, they require synchronization mechanisms like mutexes to avoid race conditions, and a bug in one thread can affect the entire process.

In practice, I would choose **multiprocessing** when isolation, security, or fault tolerance is the priority, and **multithreading** when tasks need to share data efficiently and achieve high concurrency with lower overhead.
